HumanGL

Rapport de toutes les modifications du projet

Dépôt TuturPr/HumanGl · branche main · 13/10/2026

Indicateur	Valeur
Pull requests mergées	6
Commits	41
Code (src, tests, Makefile)	+7731 / −1593 lignes
Dépôt complet (dépendances inutiles comprises)	2477 fichiers, +8094 / −468651 lignes
Tests unitaires	375 tests, 29 suites, tous réussis

1. Résumé

Point de départ. Le dépôt contenait un moteur hérité du projet Scop :

- les transformations reposaient sur GLM, alors que le sujet impose des matrices maison ; la partie obligatoire n'était donc pas validable ;
- il embarquait de nombreuses dépendances inutilisées : `stb` presque entier, un chargeur OBJ, ainsi que la documentation, les exemples et tous les backends d'ImGui ;
- les animations (idle, walk, jump, disco) étaient mêlées au code du modèle.

Point d'arrivée.

- **Partie obligatoire complète** :
  - bibliothèque mathématique et pile de matrices maison ;
  - modèle hiérarchique à une seule fonction de dessin ;
  - repositionnement automatique quand on change la taille d'un membre ;
  - marche, saut et position immobile.
- **Animations isolées du moteur de rendu**, dans un module sans aucune dépendance OpenGL.
- **Bonus** :
  - parties supplémentaires (cou, mains, pieds) ;
  - trois nouvelles animations : Disco, Kung-fu et **Thomas flare** (breakdance) ;
  - interface Dear ImGui pour régler chaque membre (taille, couleur) et changer d'animation à la volée, avec fondu enchaîné.
- **Qualité** :
  - mouvements fluides (splines C1, fondus) ;
  - articulations sans trou visible ;
  - limites anatomiques respectées sans bornage, avec en plus un filet de sécurité ;
  - architecture SOLID ;
  - compilation avec `-Wall -Wextra -Werror` sans aucun avertissement.
- **Tests** : une suite GoogleTest couvre chaque module sans OpenGL.

Méthode de travail.

1. Un audit complet du code, puis la suppression du code inutile.
2. Une branche et une pull request par fonctionnalité.
3. Les tests unitaires de chaque modification, écrits par un sous-agent utilisant un autre modèle (Sonnet).
4. Une relecture de chaque PR par un second sous-agent avant le merge.
5. Une vérification finale de la conformité au sujet par un sous-agent dédié.
6. Les explications de chaque choix, rédigées dans `docs/DECISIONS.md` et reprises en section 4.

Commits au format `ADD:` / `CHANGE:` / `DEL:` / `REFACTOR:`.

2. Conformité au sujet

Exigence du sujet	Réalisation	Où
Makefile	`make`, `make test`, `clean`, `fclean`, `re` ; dépendances d'en-têtes (`-MMD`)	`Makefile`
Matrices et transformations maison, OpenGL ≥ 4.0	`Vec3`, `Mat4`, `MatrixStack` ; aucun GLM, aucune fonction matricielle dépréciée ; contexte OpenGL 4.1 core	`src/math/`, `src/platform/Window.cpp`
Parties articulées par la pile de matrices	Le torse est la racine ; chaque enfant est dessiné dans le `push` / `pop` de son parent	`src/skeleton/HumanModel.cpp`
Si le torse tourne, tous les membres suivent	Rotation de la racine appliquée avant tous les enfants	`HumanModel::draw` ; tests `TorsoYaw/Pitch/RollMovesEveryPart`
Si le bras bouge, seul l'avant-bras suit	Rotation de l'épaule à l'intérieur du sous-arbre du bras uniquement	`HumanModel::drawArm` ; tests `LeftShoulderFlexMovesOnlyLeftArmChain`, `ElbowFlexMovesForearmAndHandButNotUpperArm`
Changer la taille d'un membre repositionne les parties liées	Position de chaque articulation calculée à partir de `BodyDimensions` ; l'échelle ne fuit jamais aux enfants	`BodyDimensions`, `drawPart` ; tests `LongerUpperArmRepositionsForearmAndHand`, `UpperArmScaleDoesNotLeakToForearm`
Tête, torse, 2 × (bras, avant-bras), 2 × (cuisse, jambe)	Les 10 parties obligatoires ; cou, mains et pieds en bonus, désactivables	`src/skeleton/BodyPart.hpp`
Marcher, sauter, rester immobile	Walk, Jump, Idle (touches 1, 2, 3 ou panneau)	`src/animation/clips/`
Une partie = un seul appel de fonction qui dessine une forme 1x1x1 à l'origine de la matrice courante	`HumanModel::drawPart` : seul endroit de dessin, appelé une fois par partie	tests `DrawsEachOfFifteenPartsExactlyOnceWithBonusParts`, `DrawsTenPartsWithoutBonusParts`
Membre supérieur et inférieur = deux parties distinctes	Deux `BodyPart` distincts par membre, chacun avec son appel de dessin	`BodyPart.hpp`
Bonus : plus de parties	Cou, mains, pieds	`BodyPart.hpp`, panneau ImGui
Bonus : autres mouvements	Disco, Kung-fu, Thomas flare (touches 4, 5, 6)	`src/animation/clips/`
Bonus : interface graphique	Dear ImGui : animation, vitesse, fondu, pause, taille et couleur de chaque partie, options d'affichage	`src/ui/`

Exigence du sujet	Réalisation	Où
Soutenance : changer la taille des membres facilement	À l'exécution (panneau) ou dans le code (`BodyDimensions.cpp`, une ligne par partie)	README

Vérification finale

Un sous-agent indépendant (Opus) a vérifié le projet sur un **clone neuf** de `main` (commit `1b239a7`). **Verdict : PASS.** Aucun point bloquant.

Vérification	Résultat
`make` avec `-Wall -Wextra -Werror`	OK, aucun avertissement ; un second `make` ne recompile rien
`make clean`, `fclean`, `re`	OK
`make test`	375 tests, 29 suites : tous réussis (avant et après `make re`)
Absence de GLM et des fonctions matricielles dépréciées	OK : seules deux mentions, dans des commentaires
Contexte OpenGL	4.1 core ; shaders et ImGui en `#version 410 core`
Une seule fonction de dessin par partie	OK : `drawPart` est le seul appelant de `drawUnitShape` ; chaque partie est dessinée une fois par parcours
Forme 1x1x1 à l'origine de la matrice courante	OK : boîte de −0,5 à 0,5 sur chaque axe, un seul `glDrawElements`
Rotation du torse, bras, redimensionnement	OK, avec les tests correspondants
Exécution réelle (Xvfb + Mesa)	OK : environ 1000 images en parcourant les 6 animations, **0 erreur OpenGL**
Animations isolées du rendu	OK : aucun en-tête OpenGL, GLFW, ImGui ou `render/` dans `animation/`, `skeleton/`, `math/`, `geometry/`
Bonus (parties, danses, interface)	OK

Remarques prises en compte :

- la marche sur place est volontaire ; c'est maintenant précisé dans `DECISIONS.md` ;
- la hiérarchie est parcourue plusieurs fois par image (contact au sol, ombre, corps), toujours avec un seul appel par partie ; c'est aussi documenté ;
- `PROGRESS.md` est à jour.

3. Historique : pull requests et commits

Une branche et une pull request par fonctionnalité, chacune relue par un sous-agent avant le merge. Les suppressions (DEL) ont été faites par simple commit sur main, comme convenu.

PR	Titre	Branche	Fichiers	Lignes
#1	ADD: own math library and matrix stack	feature/math-library	11	+889 / −2
#2	REFACTOR: modular engine, hierarchical skeleton and isolated animations	refactor/engine-architecture	2204	+5449 / −402686
#3	ADD: ImGui control panel (live limb sizes, colours, animations)	feature/imgui-panel	11	+455 / −5
#4	ADD: Thomas flare breakdance animation	feature/thomas-flare	9	+510 / −9
#5	ADD: Disco and Kung-fu animations	feature/dance-moves	10	+479 / −25
#6	ADD: README and design decisions	docs/decisions	3	+341 / −3

Liste des commits (du plus ancien au plus récent)

Commit	Message	Date
ebe5fdf	DEL: unused ImGui docs, examples and backends	12/10/2026 23:23
5e4752c	ADD: gitignore for build artifacts	12/10/2026 23:24
9b6d9d5	ADD: own Vec3, Mat4 and MatrixStack	12/10/2026 23:24
f7a186f	ADD: make test rule with GoogleTest	12/10/2026 23:27
64950b1	ADD: math library unit tests	12/10/2026 23:27
f9f1bb2	REFACTOR: apply math review nits	12/10/2026 23:29
14f680f	ADD: hierarchical skeleton with grounding and joint limits	12/10/2026 23:35
38fb8dc	ADD: animation module isolated from rendering	12/10/2026 23:35
2feeaef	ADD: render engine on own math with rounded unit shape	12/10/2026 23:35
8cb6bd3	REFACTOR: application loop, window and Makefile	12/10/2026 23:35
5923a90	DEL: GLM, stb, OBJ loader and legacy model code	12/10/2026 23:35
e5d900f	CHANGE: walk arms swing opposite to the legs	12/10/2026 23:43
f5b11bd	CHANGE: smoother jump timing with real flight time	12/10/2026 23:43
bea8c6b	CHANGE: joint overlap from thinner part, roll before pitch	12/10/2026 23:43
1522c17	ADD: support pinning for ground contact	12/10/2026 23:43
794add4	ADD: skeleton, animation, geometry and camera unit tests	12/10/2026 23:45
ca8850e	REFACTOR: drawPart is the single drawing function	12/10/2026 23:51
f3c2e12	CHANGE: review fixes (shadow offset, dt clamp, guards)	12/10/2026 23:51
a35850c	CHANGE: quieter Makefile output	12/10/2026 23:51
3fea3f6	ADD: tests for single draw call and review fixes	12/10/2026 23:52
b3aa35d	ADD: progress tracker	12/10/2026 23:54
c5f03aa	ADD: mirrorPart helper for symmetric edits	12/10/2026 23:56
2e73785	ADD: ImGui control panel for live body and animation edits	12/10/2026 23:56
28a5275	CHANGE: wire ImGui panel into the application	12/10/2026 23:56
a238c0a	ADD: body part and symmetric editing tests	12/10/2026 23:57
6c3fc35	CHANGE: progress tracker	12/10/2026 23:57

Commit	Message	Date
78e1978	CHANGE: abduct before flex at shoulders and hips	12/10/2026 23:58
8191846	ADD: Thomas flare breakdance animation	12/10/2026 23:58
7c34db4	CHANGE: clamp panel sliders, tooltip, includes	12/10/2026 23:59
69e9112	CHANGE: smoother flare hand switch	13/10/2026 00:08
b0761db	ADD: Thomas flare and joint order tests	13/10/2026 00:08
326daf3	CHANGE: progress tracker	13/10/2026 00:08
ba2ba6d	CHANGE: strict flare joint limit test	13/10/2026 00:26
df11886	ADD: disco and kung-fu animations	13/10/2026 00:27
7a93964	CHANGE: unlocked kung-fu punch elbow	13/10/2026 00:29
5002cef	ADD: disco and kung-fu tests	13/10/2026 00:29
eae6b81	CHANGE: progress tracker	13/10/2026 00:29
fc4f20c	CHANGE: dance test review fixes	13/10/2026 00:31
5dab80d	ADD: README and design decisions	13/10/2026 00:31
86440b0	CHANGE: doc review fixes	13/10/2026 00:33
7bb6d9b	CHANGE: defense notes and progress	13/10/2026 00:40

4. Modifications détaillées et choix de conception

Ce document explique, pour chaque partie du projet, **ce qui a été fait et pourquoi ce choix plutôt qu'un autre**. Le critère de départage a toujours été le même : la solution la plus propre, la plus lisible et la plus simple à expliquer en soutenance.

1. Audit et nettoyage du dépôt

Constat de départ. Le dépôt d'origine utilisait GLM pour toutes ses transformations, embarquait `stb` presque entier, un chargeur OBJ, la documentation, les exemples et tous les backends de Dear ImGui, ainsi que des classes VAO/VBO/EBO/Texture qui ne servaient plus.

Ce qui a été supprimé (commits `DEL:` directement sur `main`, comme convenu) : GLM, `stb`, le chargeur OBJ, l'ancien code du modèle, les fichiers ImGui inutiles (docs, exemples, backends autres que GLFW / OpenGL 3).

Pourquoi. Le sujet exige « your own matrices and transformations » : garder GLM rendait la partie obligatoire invalide. Le reste n'était appelé par aucun fichier ; le garder ne faisait qu'allonger la compilation et la lecture.

2. Bibliothèque mathématique maison (`src/math/`)

- `Vec3`, `Mat4` (stockage column-major, identique à ce qu'attend OpenGL), `MathUtils`.
- `MatrixStack` : `push`, `pop`, `translate`, `rotate`, `rotateX/Y/Z`, `scale`.

Pourquoi column-major ? `glUniformMatrix4fv` reçoit la matrice telle quelle, sans transposition : aucune conversion cachée à expliquer.

Pourquoi post-multiplier dans la pile ? C'est le comportement de l'ancienne pile OpenGL (`glPushMatrix`, `glTranslate`…). Le code se lit alors dans l'ordre de la hiérarchie : d'abord le parent, puis l'enfant. C'est ce qu'un évaluateur habitué à OpenGL s'attend à voir.

Pourquoi `pop()` lève une exception sur la matrice de base ? Un `pop` de trop est toujours un bug de hiérarchie ; mieux vaut le voir immédiatement (et le tester) que dessiner un modèle silencieusement faux.

Alternative écartée : une bibliothèque générique `Mat<N,M,T>`. Plus flexible, mais beaucoup plus longue à lire pour un besoin limité aux matrices 4x4.

3. Architecture en modules (principes SOLID)

```
math/       Vec3, Mat4, MatrixStack            (aucune dépendance)
skeleton/   Pose, BodyDimensions, HumanModel,  (dépend de math/)
            JointLimits, Character, IPartDrawer
animation/  IAnimation, KeyframeTrack,         (dépend de skeleton/Pose uniquement)
            Animator, AnimationLibrary, clips/
geometry/   forme unitaire 1x1x1                (aucune dépendance OpenGL)
render/     shaders, maillage GPU, caméra,      (seul module qui parle à OpenGL)
            ombre, SceneRenderer
ui/         ImGuiLayer, ControlPanel           (interface bonus)
platform/   Window (GLFW)
app/        Application : assemble le tout
```

Animations isolées du moteur de rendu. Une animation est une fonction pure du temps (`IAnimation::sample(t) -> Pose`). Elle ne connaît ni OpenGL, ni les dimensions du corps. Conséquences :

- on peut tester toutes les animations sans contexte OpenGL ;
- changer de moteur de rendu ne touche à aucune animation.

Application de SOLID :

- **S** (une responsabilité par classe) : `HumanModel` ne fait que parcourir la hiérarchie, `Animator` ne fait que jouer et fondre les animations, `SceneRenderer` ne fait que dessiner.
- **O** (ouvert / fermé) : ajouter une danse, c'est écrire un fichier dans `clips/` et une ligne dans `AnimationLibrary::createDefault()`. Ni l'`Animator`, ni l'interface, ni les touches clavier ne changent (elles parcourent la bibliothèque).
- **L** (substitution) : toutes les animations sont interchangeables derrière `IAnimation`.
- **I** (interfaces minimales) : `IPartDrawer` n'a qu'une méthode, `IAnimation` deux.
- **D** (inversion des dépendances) : le squelette ne dépend pas d'OpenGL ; il appelle `IPartDrawer`. Le rendu OpenGL, l'ombre portée, le calcul du contact au sol et les tests unitaires sont autant d'implémentations de cette interface.

Alternative écartée : un graphe de scène générique (nœuds, composants). Plus puissant, mais il cacherait la pile de matrices que le sujet demande justement de montrer et d'expliquer.

4. Modèle hiérarchique (`skeleton/HumanModel`)

```
Torse ─┬─ Cou ── Tête
       ├─ Bras G/D ── Avant-bras G/D ── Main G/D
       └─ Cuisse G/D ─ Jambe G/D ──── Pied G/D
```

- Le **torse est la racine** : une rotation du torse est appliquée une seule fois, avant tous les enfants, donc tout le corps suit.
- Chaque membre est dessiné dans un `push` / `pop` : quand le bras bouge, seul l'avant-bras (et la main) suit.
- **La position de chaque articulation est calculée à partir des dimensions du parent** (`BodyDimensions`). Modifier la longueur d'un membre repositionne automatiquement les parties qui en dépendent, sans aucune valeur codée en dur.
- **L'échelle d'une partie est poussée puis retirée autour de son propre dessin uniquement** : elle ne se propage jamais aux enfants. C'est l'erreur classique d'une pile de matrices, et elle est couverte par des tests.

Une seule fonction de dessin. `HumanModel::drawPart` est le seul endroit où une partie est dessinée, et elle est appelée exactement une fois par partie. Elle dessine la forme 1x1x1 à l'origine de la matrice courante, mise à l'échelle. Un test vérifie qu'il y a exactement un appel par partie.

Plusieurs parcours par image, toujours un appel par partie. À chaque image, la hiérarchie est parcourue plusieurs fois : pour trouver le contact au sol (ce parcours ne dessine rien), pour l'ombre puis pour le corps. Chaque parcours appelle `drawPart` exactement une fois par partie ; seul l'`IPartDrawer` change.

Ordre des rotations aux épaules et aux hanches. L'abduction est appliquée d'abord, dans le repère de l'os ; la flexion vient ensuite, autour de l'axe X du parent, et fait pivoter vers l'avant le membre déjà écarté. Dans le code, `rotateX(flexion)` précède donc `rotateZ(abduction)` (la pile post-multiplie). Un bras écarté puis porté vers l'avant déplace ainsi la main sur le côté, comme une vraie épaule ; c'était nécessaire pour le Thomas flare (main sous la poitrine). La suite de tests `HumanModelRotationOrder` couvre cet ordre.

5. Articulations invisibles et forme unitaire

Problème. Avec des cubes stricts posés bout à bout, un genou ou un coude plié laisse apparaître un trou ou un angle vif.

Solution retenue.

1. Chaque segment de membre est **prolongé au-delà de ses articulations** d'une demi-épaisseur de la partie la plus fine. Deux segments voisins se recouvrent toujours, quel que soit l'angle.

2. La forme unitaire est une **boîte 1x1x1 aux arêtes légèrement arrondies** (`geometry/RoundedBox`). Elle reste une forme géométrique 1x1x1 dessinée par un seul appel, comme l'exige le sujet. Le recouvrement des extrémités arrondies cache la jointure.

Alternatives écartées : des sphères aux articulations (cela ajoute des parties que le sujet ne demande pas et un deuxième appel de dessin par articulation) ; des maillages déformés par skinning (hors sujet : le projet porte sur la pile de matrices, pas sur le skinning).

6. Limites articulaires (`skeleton/JointLimits`)

Chaque articulation a une plage anatomique en degrés : un coude ou un genou va de 0 à 150° et ne se plie jamais à l'envers, le cou et les poignets sont limités, etc. Le **torse est libre** : le corps entier peut tourner ou basculer (salto, flare), seules les articulations entre deux parties sont limitées.

Double protection :

- l'`Animator` borne chaque pose produite (`JointLimits::clamp`). Une animation mal réglée ne peut donc jamais produire un mouvement impossible à l'écran ;
- les tests vérifient que chaque animation reste déjà dans les limites sans bornage (strictement pour Disco, Kung-fu et Thomas flare ; à 2° de dépassement de spline près pour les autres), pour que le bornage ne soit qu'un filet de sécurité.

7. Animations fluides (`animation/`)

- `KeyframeTrack` : interpolation Hermite cubique avec tangentes de Catmull-Rom, en boucle. La position **et la vitesse** sont continues partout, même au raccord de la boucle : pas d'à-coups. Alternative écartée : l'interpolation linéaire, qui provoque des cassures de vitesse visibles à chaque image clé.
- **Une `Pose` est un simple vecteur de nombres** qui supporte `+` et `*`. Les splines, les fondus et les mélanges sont donc tous de simples sommes pondérées.
- `Animator` : changer d'animation déclenche un **fondu enchaîné** depuis la pose courante (0,4 s par défaut, réglable), sans saut brusque. La vitesse et la pause n'agissent que sur l'horloge de l'animation.
- `Character` : avant le dessin, le modèle est parcouru une fois pour trouver son point le plus bas, qu'on pose sur le sol. Les genoux pliés, les jambes redimensionnées ou les mains au sol restent ainsi en contact avec le sol sans que les animations ne connaissent aucune dimension. Le support pinning garde un appui (la main du flare) fixe au sol au lieu de le faire glisser.

Animations obligatoires

- **Idle** : respiration et léger balancement.
- **Walk** : marche sur place, volontairement, pour que le personnage reste au centre de la caméra. Le cycle est décrit pour une seule jambe et un seul bras ; l'autre côté joue la même courbe décalée d'une demi-période (une seule source de vérité). Les bras se balancent à l'opposé des jambes.
- **Jump** : flexion, impulsion, vol avec un vrai temps de vol, réception amortie.

Animations bonus

- **Disco** : le célèbre « doigt vers le ciel puis en travers du corps », avec un balancement des hanches. Il est suivi de moulinets d'avant-bras et de rebonds des genoux sur un tempo de 120 BPM.
- **Kung-fu** : un kata. Position du cavalier, coup de poing droit, coup de poing gauche (le torse tourne avec le coup, la tête reste face à l'adversaire), puis coup de pied avant (armé, frappe, réarmé) et retour en garde. Le coude du coup de poing garde 5° de flexion : la spline ne peut ainsi jamais passer sous la limite de 0°, et un poing réel ne se verrouille pas non plus.

- **Thomas flare** (breakdance) : animation procédurale. Chaque canal est une fonction de l'angle de rotation, ce qui garantit une boucle parfaite. Le corps est horizontal, face au sol, et porté par un bras tendu à la fois, la main d'appui fixée au sol. Les jambes, en grand écart, balaient des cercles. Le changement de main est réglé pour qu'aucune articulation ne bouge de plus de 6° par image à 120 Hz. Pourquoi procédural plutôt que des images clés ? Le flare est un mouvement circulaire continu. Le décrire par l'angle de rotation est plus court, plus lisible et ne peut pas « décrocher » au raccord de la boucle.

8. Moteur de rendu (`render/`)

- **OpenGL 4.1 core profile** (le sujet demande au moins la 4.0 ; 4.1 est aussi la version maximale disponible sur macOS, donc le projet tourne sur les postes 42).
- **RAII** pour les shaders et les maillages GPU : aucune fuite de ressource, aucun `glDelete*` oublié.
- **Éclairage simple, sol quadrillé et ombre portée plane** (stencil) : ils donnent la profondeur nécessaire pour juger visuellement un saut ou un contact au sol, sans complexité inutile.
- **Caméra orbitale** à la souris : clic gauche pour tourner, molette pour zoomer.

Alternative écartée : le shadow mapping. Il est plus réaliste, mais aussi beaucoup plus de code, pour un sol plat où une ombre plane est exacte.

9. Interface graphique bonus (`ui/`)

Dear ImGui, pour tout modifier **à la volée** :

- choix de l'animation, vitesse, durée du fondu, pause ;
- largeur, longueur et épaisseur de chaque partie. Les enfants se repositionnent automatiquement. Une option applique la modification aux deux côtés à la fois (`mirrorPart`) ;
- couleur de chaque partie, remise à zéro des tailles et des couleurs ;
- affichage des parties bonus (cou, mains, pieds), fil de fer, ombre, sol ; réinitialisation de la caméra.

`ImGuiLayer` possède le contexte ImGui (RAII) et `ControlPanel` ne fait que l'interface. Le clavier et la souris ne vont à la scène que lorsque ImGui ne les utilise pas.

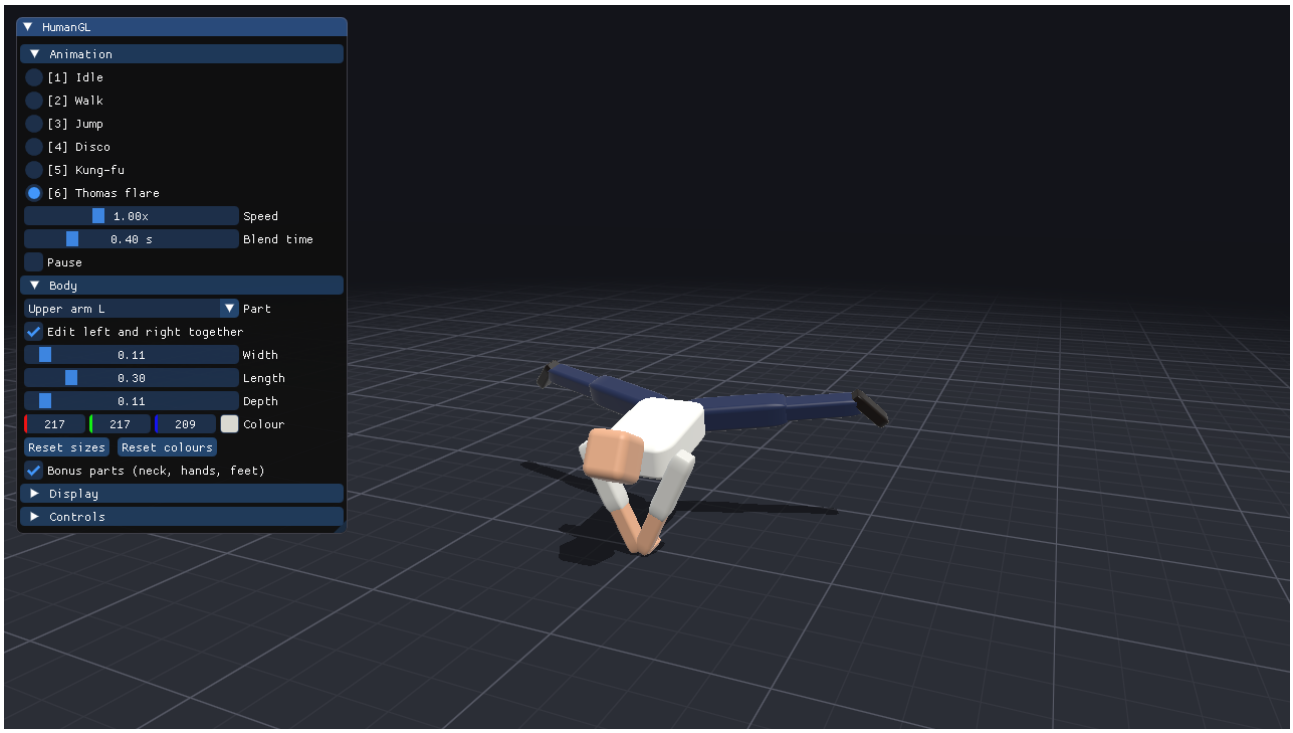
Pourquoi Dear ImGui ? Le code reste immédiat (une fonction par section), il n'y a pas d'état d'interface à synchroniser, et c'est la référence des outils de réglage temps réel.

10. Tests et processus

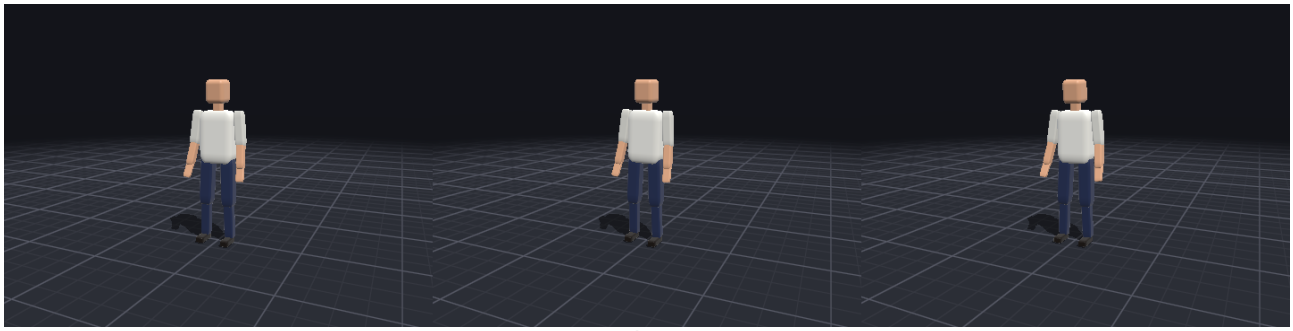
- **GoogleTest**, lancé par `make test`. Seuls les modules sans OpenGL sont liés aux tests : maths, squelette, animations, géométrie, caméra.
- Les tests vérifient des **propriétés physiques**, pas des valeurs recopiées :
  - un appel de dessin par partie ;
  - l'échelle ne fuit pas vers les enfants ;
  - les limites articulaires sont respectées sans bornage ;
  - les animations bouclent et restent fluides d'une image à l'autre ;
  - les pieds restent au sol ;
  - la main d'appui du flare ne glisse pas.
- Les tests ont été écrits par un sous-agent sur un **autre modèle** (Sonnet). Chaque PR a été relue par un sous-agent avant le merge, et un dernier sous-agent a vérifié la conformité au sujet en fin de projet.
- Commits au format `ADD:` / `CHANGE:` / `DEL:` / `REFACTOR:`, une branche et une PR par fonctionnalité.

5. Captures

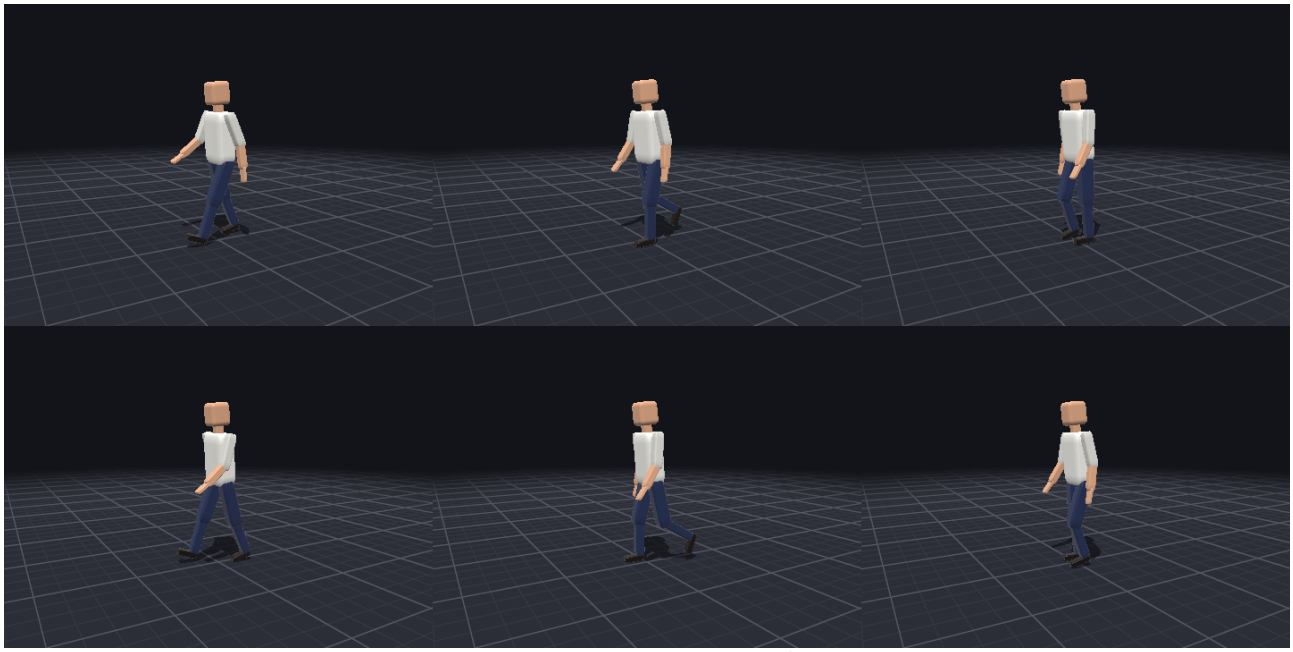
Rendus réels de l'application (OpenGL 4.1 core, Mesa, hors écran).



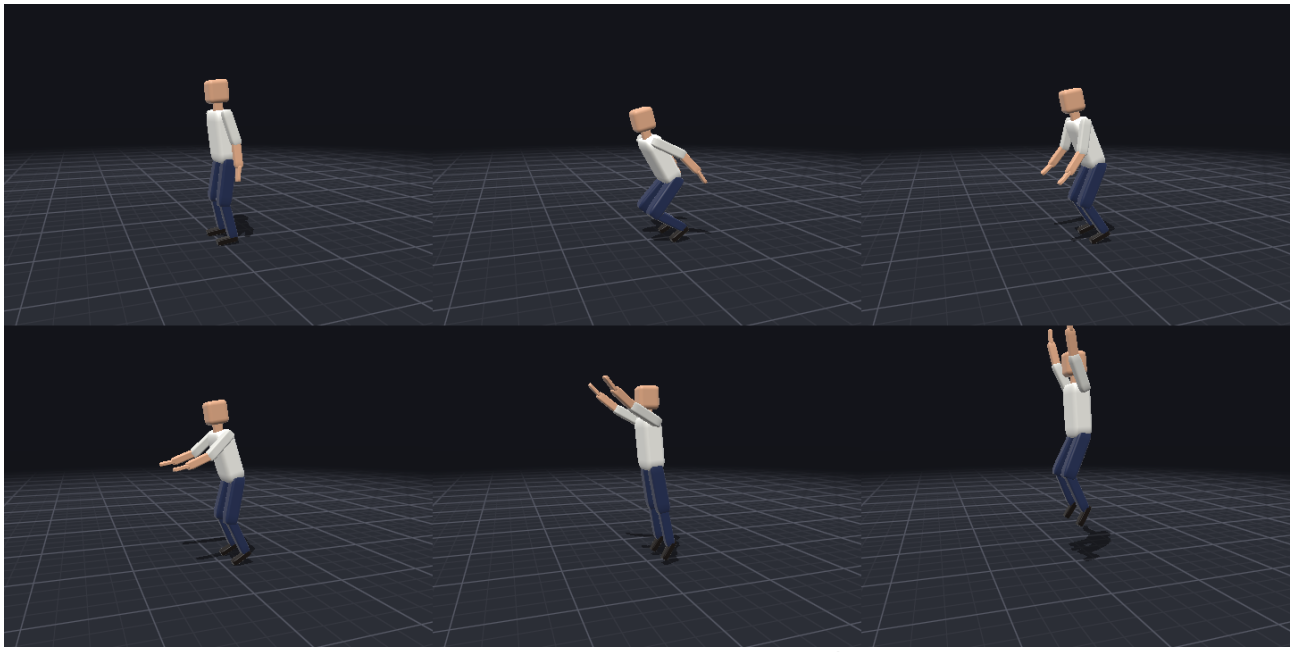
L'application et son panneau ImGui (animation, vitesse, fondu, taille et couleur de chaque partie).



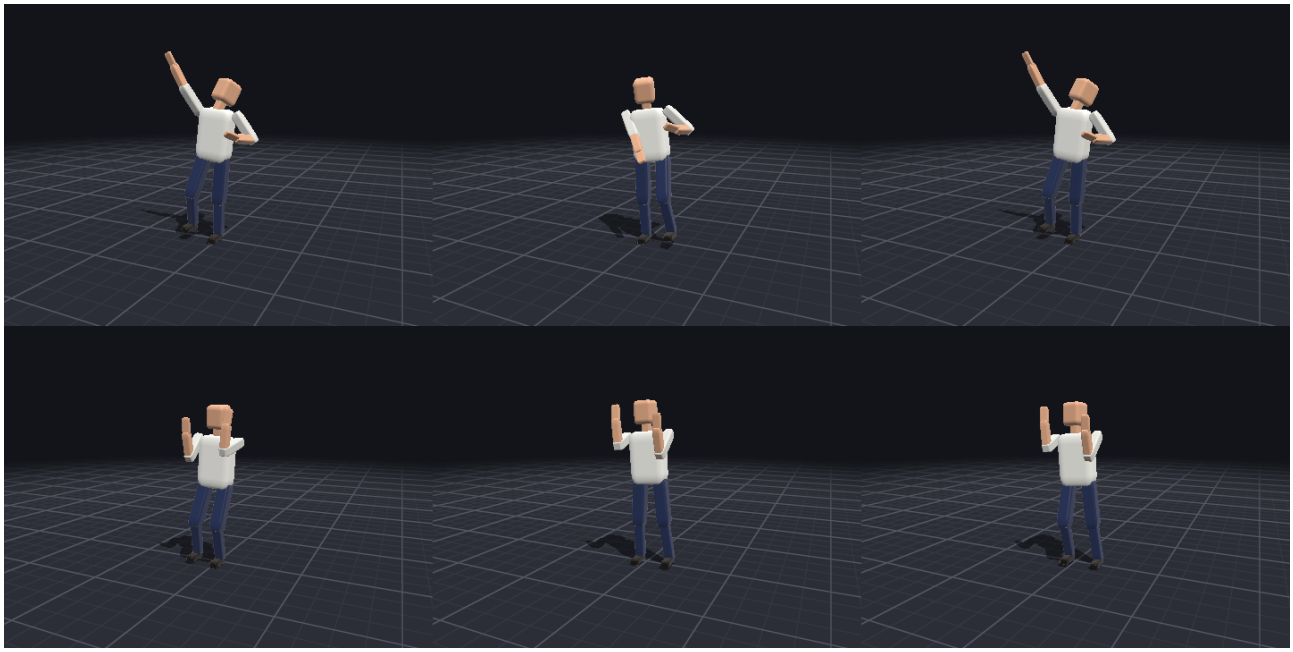
Idle : respiration et léger transfert de poids.



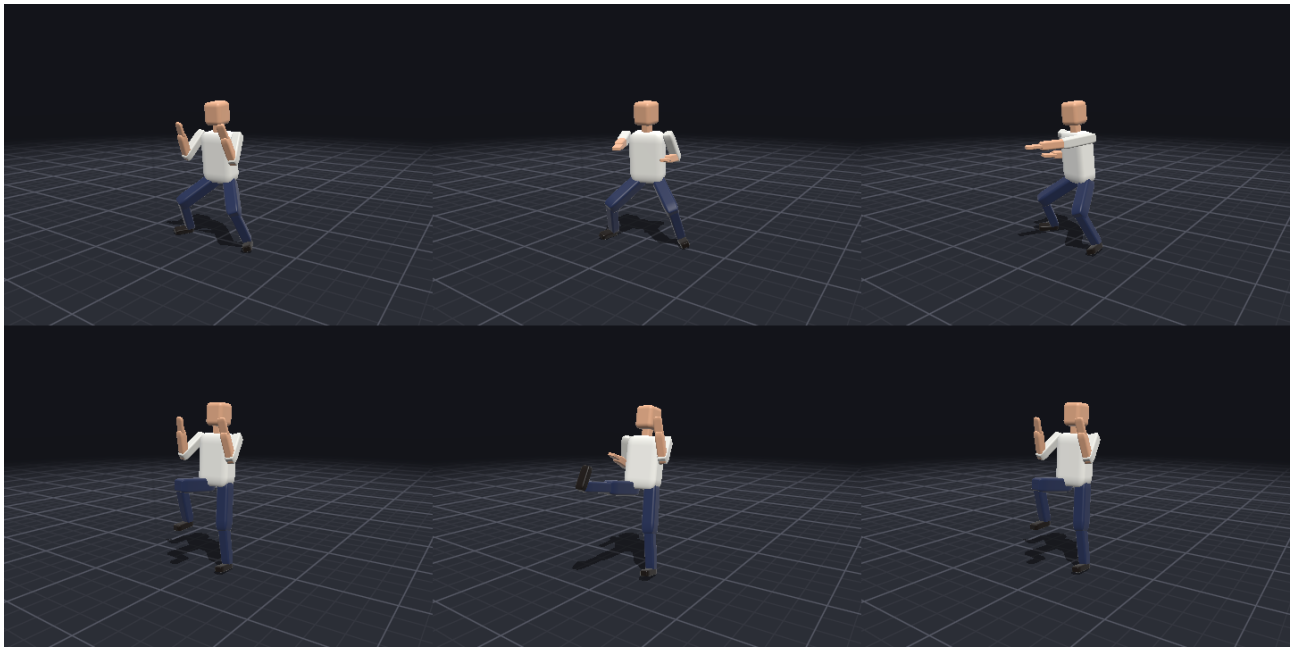
Walk : un cycle de marche, bras opposés aux jambes.



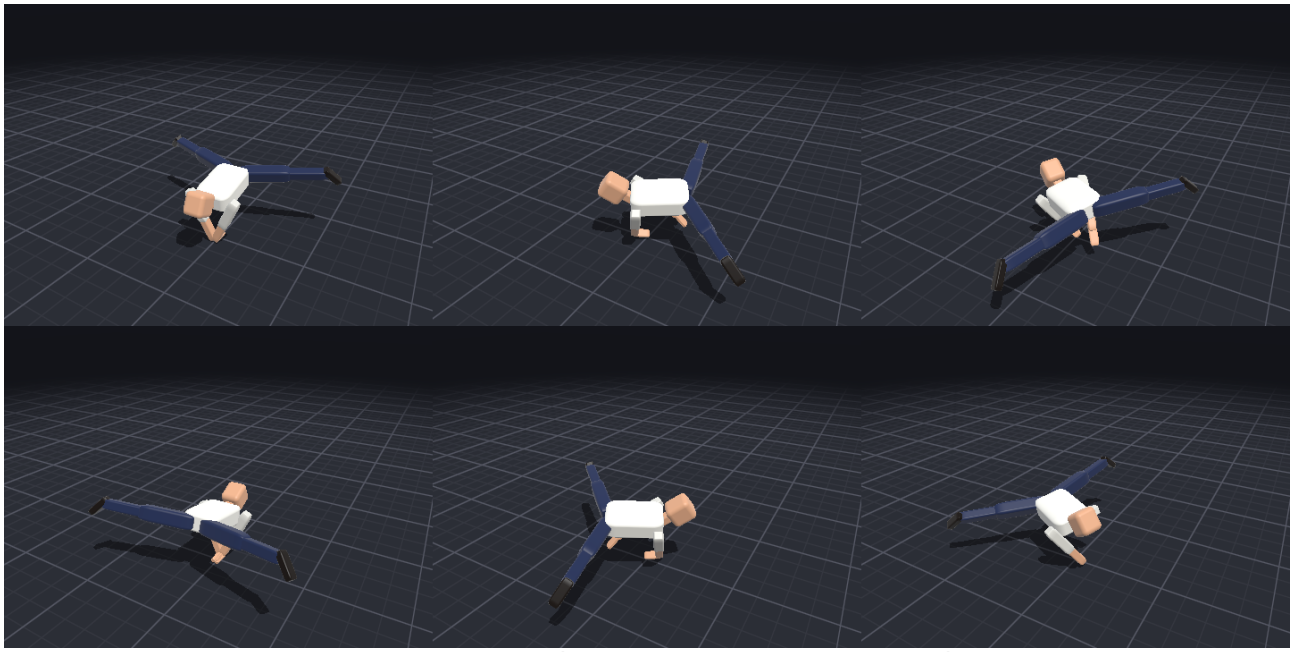
Jump : flexion, impulsion, vol balistique, réception.



Disco : doigt vers le ciel, puis en travers du corps ; moulinets et rebonds.



Kung-fu : position du cavalier, coups de poing, coup de pied avant.



Thomas flare : corps horizontal porté par une main, jambes en grand écart qui balaient.

6. Tests unitaires

375 tests répartis en 29 suites, lancés par `make test` : **tous réussis**. Les tests ont été écrits par un sous-agent utilisant un autre modèle (Sonnet). Ils vérifient des propriétés physiques plutôt que des valeurs recopiées.

Suite	Tests	Ce qui est vérifié
Vec3	16	opérations vectorielles : somme, produit scalaire et vectoriel, normalisation
MathUtils	10	conversions d'angles, fonctions utilitaires
Mat4	38	identité, produit, translation, rotation, échelle, perspective, lookAt, disposition column-major
MatrixStack	18	push / pop, ordre des transformations, pop de la base interdit
RoundedBox	19	forme unitaire : dimensions 1x1x1, centrage, normales, paramètres invalides
Pose	10	addition, mise à l'échelle et interpolation des poses
JointRange	5	bornage et appartenance d'un angle à une plage
JointLimits	8	bornage complet d'une pose, coude et genou jamais à l'envers
BodyDimensions	7	tailles par défaut, symétrie, taille minimale
BodyPart	7	noms, côtés, parties bonus, parties miroir
BodyPartEditing	4	modification symétrique gauche / droite
HumanModel	46	un appel de dessin par partie, propagation des rotations, repositionnement après redimensionnement, l'échelle ne fuit pas
HumanModelRoot	7	rotations et translation de la racine (torse)
HumanModelOverlap	7	recouvrement aux articulations (aucun trou visible)
HumanModelDrawCalls	2	un seul site d'appel de dessin
HumanModelShoulders	3	position des épaules selon la taille du torse
HumanModelRotationOrder	9	abduction puis flexion aux épaules et aux hanches
Character	17	contact au sol, point le plus bas posé sur le sol
CharacterContact	6	calcul du point d'appui
CharacterPinning	8	appui fixé au sol (main du flare)
KeyframeTrack	22	spline Hermite / Catmull-Rom : passage par les clés, continuité, boucle
Animator	27	fondu enchaîné, vitesse, pause, bornage articulaire
Clips	24	toutes les animations : limites, fluidité, boucle, pieds au sol
ThomasFlare	10	corps porté par les mains, main d'appui sans glissement, rotation complète
Disco	8	doigt au-dessus de la tête, balancement des hanches, pieds au sol
KungFu	8	coup de pied à hauteur de hanche, torsion du torse, tête face à l'adversaire
OrbitCamera	16	orbite, zoom, bornes de la caméra
PlanarShadow	8	projection de l'ombre sur le sol
BodyPalette	5	couleurs par partie, remise à zéro